

ACC Connector — Databricks Token Refresh

(Pattern 2 / U2M)

Status: Implemented and validated

Scope: User-to-Machine (U2M) login only — not M2M service principal

Date: June 2026

One-line summary

We fixed Databricks **403 Forbidden** errors that appeared after ~1 hour by saving a refresh token at login and automatically renewing the access token before bootstrap, sync, and catalog calls.

The problem

What	Detail
Symptom	Sync and Databricks API calls fail with HTTP 403 after about 60 minutes
Root cause	Login stored only the access token — no way to renew it
Impact	Users had to disconnect and log in again to keep working

The solution

Step	What we did
1. Login	Save access token + refresh token (OAuth offline_access scope)
2. Auth helper	New <code>get_valid_dbx_token()</code> — returns a valid token; refreshes if needed
3. Bootstrap	Uses fresh token before provisioning

4. Sync	Uses fresh token before every Databricks API call
5. Tests	4 unit tests cover refresh logic (all passing)

No new SDK required. Uses existing `DatabricksClient` + REST APIs (Pattern 2).

Why we did not use the Databricks SDK (or SDK + REST mix)

We evaluated the official `databricks-sdk` and a **hybrid** setup (SDK for some calls, our REST client for others). Pattern 2 — refresh in one place, then pass the token to our existing client — was the better fit.

	<code>get_valid_dbx_token</code>	SDK (<code>WorkspaceClient</code>)
Reads SQLite?	✓ Yes	✗ No
Knows <code>user_id</code> ?	✓ Yes	✗ No
Knows if token expired?	✓ Yes (checks <code>expires_at</code>)	✗ No
Uses <code>refresh_token</code> ?	✓ Yes	✗ No
Creates new access token?	✓ Yes (when expired)	✗ No
Calls Databricks APIs?	✗ No	✓ Yes

Reason	Explanation
Refresh is our problem, not the SDK's	The 403 issue was expired tokens , not missing API wrappers. The SDK does not store per-user OAuth tokens in our database. We still need <code>get_valid_dbx_token()</code> — with or without the SDK.
We already have a working REST client	<code>DatabricksClient</code> already covers bootstrap, sync, pipelines, volumes, SQL, and catalogs. Adding the SDK would duplicate what we have, not replace the broken part.

Hybrid = two stacks, one auth bug	SDK + REST together means two HTTP clients, two error-handling paths, and risk that one path refreshes tokens and the other does not. Harder to debug and test.
U2M does not map cleanly to SDK defaults	The SDK is built for PAT, env vars, or service principal. Our flow is multi-user browser OAuth with tokens in <code>connector.db</code> — that custom logic stays in the connector either way.
Less change, same outcome	Pattern 2 touches login + one auth helper + call sites. Full SDK adoption would mean rewriting many services for little gain on token expiry.
Proven in production	Full sync validated (263 files, workflow SUCCESS) with Pattern 2 — no SDK dependency required.

Bottom line: The SDK is useful for greenfield apps. For this connector, the fix was **token refresh + reuse existing `DatabricksClient`**. A full or partial SDK migration would add complexity without solving the root cause faster.

How it works (simple flow)

ONCE – User connects Databricks in the browser → OAuth login → Save `access_token` + `refresh_token` to `connector.db` EVERY – Bootstrap / Sync / Catalogs → Call `get_valid_dbx_token(user_id)` → Token still good? → Use it (no extra HTTP call) → Token expiring soon? → POST refresh to Databricks → Save new tokens → Use new token → Call Databricks APIs with valid `access_token`

Two “60” values (easy to confuse)

Name	Value	Meaning
Access token lifetime	~60 minutes	How long Databricks keeps each access token valid
Refresh buffer	60 seconds	We renew 1 minute before expiry — not 60 minutes

Example timeline

- **7:07 PM** — User connects Databricks → tokens saved, expires ~8:07 PM

- **7:12 PM** — Sync runs → token still fresh → **no refresh**, sync OK
- **~8:06 PM** — Next API call → token near expiry → **auto refresh** → sync continues OK

Files changed

Area	File	Change
Auth	backend/utils/databricks_auth.py	get_valid_dbx_token() + refresh logic
Login	backend/routes/databricks_routes.py	save refresh_token on OAuth callback
Bootstrap	backend/routes/bootstrap_routes.py	use get_valid_dbx_token()
Sync	backend/services/sync/sync_orchestrator.py	use get_valid_dbx_token()
UI	static/app.js	AWS workspace URL detection
Tests	tests/test_databricks_auth.py	4 unit tests (mock, no live Databricks)

Not in scope: M2M service principal, databricks-sdk, Zerobus (ENABLE_ZEROBUS=false).

Configuration required

Add to .env:

```
DATABRICKS_OAUTH_SCOPE=all-apis offline_access
```

offline_access is required so Databricks returns a **refresh_token** at login.

How to verify

Quick test (developers)

```
cd acc-connector python -m unittest tests.test_databricks_auth -v
```

Expected: **4 tests, OK**

Database check (after login)

Check	Good value	Meaning
has_refresh_token	1	Login saved refresh token — renewal can work
minutes_until_expiry	positive number	Token still valid
minutes_until_expiry	negative	Token expired — next sync should auto-refresh

Production proof

1. Run the app and trigger **Sync** or **Catalogs** after token expiry
2. In logs, look for: Refreshing Databricks access token for user ...
3. Sync should complete without 403

Scenario	What you see in logs
Token still fresh	Sync starts — no refresh message (normal)
Token expired, refresh OK	Refresh message → sync continues
Refresh rejected	User must connect Databricks again

AWS workspaces only

Supported URL patterns:

- `https://dbc-XXXX.cloud.databricks.com`
- `https://dbc-XXXX.databricks.us` (GovCloud)

Azure/GCP workspace detection was removed for this deployment.

Acceptance checklist

- [x] `get_valid_dbx_token` with 60-second early refresh buffer
- [x] Login saves `refresh_token` (`offline_access`)
- [x] Bootstrap and Sync use `get_valid_dbx_token`

- [x] Unit tests: 4/4 pass
- [x] Full sync validated (263 files, workflow SUCCESS)
- [x] DB confirms has_refresh_token=1
- [] Manual confirm: refresh after expiry (run sync when minutes_until_expiry is negative)

For tickets / status updates

U2M (Pattern 2) complete — get_valid_dbx_token refresh flow with offline_access and refresh token storage; bootstrap, sync, and catalogs validated; DB shows has_refresh_token=1; unit tests 4/4 pass.

Copy this page into Microsoft OneNote: open this file, select all, paste into a new OneNote page. Tables and headings will carry over cleanly.